

# SQAT — MID EXAM STUDY PACK

---

---

Software Quality & Testing · CSC 4133  
Chapters 1–6

Complete self-contained study pack  
Cram sheet · Full notes · Mock exam + key · 100 rapid-fire · Confusion tables · Memorize sheet  
Built strictly from the lecture slides

## Contents

---

1. Cram Sheet `00_CRAM_SHEET.md`
2. Chapter Notes `01_CHAPTER_NOTES.md`
3. Mock Exam `02 MOCK_EXAM.md`
4. Answer Key `03_ANSWER_KEY.md`
5. Rapid Fire (100 Q&A) `04_RAPID_FIRE.md`
6. Confusion Tables `05_CONFUSION_TABLE.md`
7. Memorize This `06_MEMORIZE_THIS.md`

**Note on slide files:** the .pptx file names are shifted +1 from the chapter inside. File “Ch.02” = Chapter 1, “Ch.03” = Chapter 2, ..., “Ch.06 Unit Testing” = Chapter 6. This pack uses the content chapter numbers.

# 00 — CRAM SHEET (last-hour read)

**Course:** Software Quality and Testing (SQAT) — CSC 4133 · **Exam:** Mid, Ch 1–6 **Textbooks:** Naik & Tripathy · Jeff Tian

**File-name note:** the .pptx file names are shifted +1 from the chapter inside. Ch.02 file = **Chapter 1**, Ch.03 file = **Chapter 2**, ... Ch.06 Unit Testing file = **Chapter 6**. This pack uses the *content* chapter numbers (what the exam uses).

## CH 1 — Basics of Software Quality Assurance

- **Software** = programs + procedures + documentation + data. Two types: **Generic (Buy → COTS)**, **Customized (Build → bespoke)**.
- **Quality (ISO/IEC 9126):** totality of functionality + features that satisfy **stated or implied needs**.
- **Quality (IEEE 610):** degree to which a component/system/process meets specified requirements + user needs.
- **3 project challenges: Time, Cost, Scope.**
- **Famous defects: Therac-25** (radiation machine, AECL 1986, deaths from overdose — software bug) · **Ariane 5** (4 June 1996, exploded 37 s after launch — most expensive computer bug).

|              | SQA                                                        | SQC                                                       |
|--------------|------------------------------------------------------------|-----------------------------------------------------------|
| Aim          | <b>Prevent</b> defects                                     | <b>Identify &amp; resolve</b> defects                     |
| Program run? | <b>Does NOT execute</b> program                            | <b>Always executes</b> program                            |
| Who          | <b>All team members</b>                                    | <b>Only testing team</b>                                  |
| Maps to      | <b>Verification</b> (doing things right, process-oriented) | <b>Validation</b> (results as expected, product-oriented) |
| System type  | Failure <b>prevention</b> system                           | Failure <b>detection</b> system                           |
| Scope        | Full <b>SDLC</b>                                           | Software <b>testing life cycle</b>                        |

- **SQC review activities:** Requirement, Design, Code, Deployment Plan, Test Plan, Test Cases review.
- **Static analysis (manual):** examine code/docs, reason over all behaviours → code review, inspection, algorithm analysis. Slow.
- **Dynamic analysis (automated):** actually **execute** program to expose failures. → complementary to static.
- **Error → Fault → Failure → Defect** (memorize cold):
- **Error** = human action producing an incorrect result (injects a fault).
- **Fault** = incorrect step/process/data definition in the program (the underlying condition).
- **Failure** = inability of system to perform required function.
- **Defect** = behavioural deviation from requirement/spec; collectively = **"bugs"**.
- **SqE 3 stages: Planning → Execution → Decision.**
- **Complete/Exhaustive testing** ≈ impossible: input domain too large; too complex; can't build all environments.

## CH 2 — Software Quality

- **Perspectives: External/Consumer** (customers, users) = **black box** · **Internal/Producer** (developers, testers, managers) = **white/clear box** · **Other** (3rd party).
- **5 Views of quality (Kitchenham & Pfleeger)** — mnemonic **"My Uncle Makes Pretty Vases"**:

| # | View                 | One line                                                                                 |
|---|----------------------|------------------------------------------------------------------------------------------|
| 1 | <b>Mystical</b>      | Quality recognized by experience, not defined; good quality "stands out"                 |
| 2 | <b>User</b>          | Meets user needs/expectations; good if satisfies many users                              |
| 3 | <b>Manufacturing</b> | Conformance to process/requirements; "right the first time" → <b>CMM &amp; ISO 9001</b>  |
| 4 | <b>Product</b>       | Good internal properties → good external properties                                      |
| 5 | <b>Value-based</b>   | Willingness to pay; <b>excellence (quality) + worth (value)</b> ; cost/quality trade-off |

- **Why measure quality:** Baseline · Quality improvement (based on cost) · Know present level for future planning.
- **Quality FACTOR** = behavioural characteristic of a system (external). Examples: **Correctness, Reliability, Efficiency, Performance, Integrity/Security, Usability**.
- **Quality CRITERION** = attribute of a factor, related to development (internal). Examples: **Modularity, Testability, Maintainability, Reusability, Flexibility, Portability, Interoperability**.
- **Model: McCall's** Quality Factors & Criteria.
- **Reliability** = probability software executes **without failure for a specific period of time**.
- **ISO-9126** — most influential; hierarchical (characteristics + non-overlapping sub-characteristics). **6 top-level characteristics** — mnemonic "**Fat Rabbits Usually Eat More Peas**":

| Functionality  | Reliability         | Usability     | Efficiency       | Maintainability       | Portability |
|----------------|---------------------|---------------|------------------|-----------------------|-------------|
| what is needed | functions correctly | effort to use | resources needed | correct/improve/adapt | env → env   |

**TRAP:** Security is **NOT** an ISO-9126 top-level characteristic. Reliability is a **factor**; Modularity is a **criterion**.

## CH 3 — Maturity Models

- **Software process** = set of activities to develop products. Benefits: **repeatable, measurable, improvable**.
- **Process tasks:** Requirements Analysis, Design/Modeling, Coding, Testing, Implementation/Integration, Operation/Maintenance, Documentation.
- **Why improve a process:** **Quality, Lead Time, Cost**.
- **3 models:** **CMM** = evaluate **development** process · **TPI** = **improve** testing process · **TMM** = **evaluate** testing process.

**CMM — 5 levels** (mnemonic "**I Really Don't Manage Overtime**");

| L | Name              | Meaning                                                           |
|---|-------------------|-------------------------------------------------------------------|
| 1 | <b>Initial</b>    | Disorganized/chaotic; depends on individuals; not repeatable      |
| 2 | <b>Repeatable</b> | Basic project management; successes repeatable                    |
| 3 | <b>Defined</b>    | Own standard process; documentation, standardization, integration |
| 4 | <b>Managed</b>    | Monitors & controls via <b>data collection &amp; analysis</b>     |
| 5 | <b>Optimizing</b> | Constant improvement via feedback + innovation                    |

**TRAP:** Level 4 = **Managed**, Level 5 = **Optimizing** (not reversed).

**TMM — 5 levels** (Ilene **Burnstein**) (mnemonic "**I Prefer Increasingly Mature Orgs**");

| L | Name                                             | Key idea                                                                 |
|---|--------------------------------------------------|--------------------------------------------------------------------------|
| 1 | <b>Initial</b>                                   | No goals; testing after code; ad hoc; not tracked                        |
| 2 | <b>Phase Definition</b>                          | Testing+debugging goals; initiate <b>test planning</b>                   |
| 3 | <b>Integration</b>                               | Establish <b>test group</b> ; training; integrate testing into lifecycle |
| 4 | <b>Management and Measurement</b>                | Org-wide review; test management; evaluate quality                       |
| 5 | <b>Optimization / Defect Prevention &amp; QC</b> | Defect prevention; statistical QC; process optimization                  |

- **TMM level parts:** Maturity goals, Supporting maturity goals, **ATRs** (Activities, Tasks, Responsibilities).
- **TPI — improve test process (4 steps):** ① Determine area ② Evaluate current state (baseline) ③ Identify next state + means ④ Implement changes.

## CH 4 — Software Quality Assurance

- **QA = correctness aspect, dealing with defects. 3 generic categories** (mnemonic "**Prevent, Reduce, Contain**"):

| Category                  | What it does                           | Example                               |
|---------------------------|----------------------------------------|---------------------------------------|
| <b>Defect Prevention</b>  | Stop faults being <b>injected</b>      | default values, input options         |
| <b>Defect Reduction</b>   | Detect & <b>remove</b> injected faults | inspection, testing                   |
| <b>Defect Containment</b> | Contain failures / limit damage        | <b>auto-save</b> , exception handling |

- **Defect prevention — 2 ways: Error source removal** (education/training — people-based) · **Error blocking** (block wrong actions, e.g. can't divide by 0, data validation). Also process improvement / formal methods.
- **Defect reduction techniques: Inspection** (static — informal walkthroughs vs formal inspections) · **Testing** · Boundary value analysis · Control/data flow analysis · Simulation/Prototyping.
- **Defect containment — 2 ways: Fault tolerance** (Recovery = rollback & redo; **NVP** = N-Version Programming) · **Safety assurance**.
- **Safety definitions (memorize):**
- **Safety** = accident-free · **Accident** = failure with severe consequences · **Hazard** = pre-condition to an accident.
- **Safety Assurance** = hazard analysis, hazard elimination/reduction/control, damage control.
- **Process variations:** Waterfall · Iterative (QA in increments) · Spiral (QA + risk mgmt) · Agile (XP, TDD, pair programming) · Maintenance (defect handling).
- **DC → V&V mapping:** Unit = **Verification**; Beta/Operation = **Validation**; Acceptance = **both, more validation**.

**TRAP:** Prevention is **never 100% effective** → that's why reduction & containment exist. NVP & auto-save = **containment**, not prevention.

## CH 5 — Software Quality Engineering (SQE)

- **SQE** = meet/exceed quality expectations via appropriate QA activities while **minimizing cost & risk** under constraints.
- **Generic testing process (3 phases): Pre-QA** (planning) → **In-QA** (execution) → **Post-QA** (measurement/assessment/improvement).
- **Pre-QA quality goals:** identify quality perspective/expectation · select direct quality measures (quantified) · assess expectations vs. cost.

| Term              | Level                 | One line                                                    |
|-------------------|-----------------------|-------------------------------------------------------------|
| <b>Test Plan</b>  | <b>High-level</b> doc | objectives, scope, approach, resources, schedule of testing |
| <b>Test Case</b>  | <b>Low-level</b> doc  | one input/action + expected result to check a feature       |
| <b>Test Suite</b> | <b>Macro-level</b>    | collection of test cases run until stopping criteria        |

- **Regression testing** = reuse test cases from earlier versions (**no new cases designed**).
- **Post-QA** does **not** mean "after QA finishes" — runs **parallel** to QA.
- **Testing team models:**
- **Vertical** = organized **around one product** (dedicated people, many tasks for that product).
- **Horizontal** = **one kind of testing** across many products.
- **Mixed** = both (large organizations).

## CH 6 — Software Testing

- **Testing** = execute software + observe behaviour; if failure → locate & fix fault; else gain confidence.
- **7 Principles** — mnemonic "**Please Eat Every Delicious Pizza Carefully, Alright**":

| # | Principle                        | One line                                       |
|---|----------------------------------|------------------------------------------------|
| 1 | <b>Presence not absence</b>      | shows defects <b>exist</b> , never proves none |
| 2 | <b>Exhaustive impossible</b>     | use risk/time/cost/priority                    |
| 3 | <b>Early testing</b>             | earlier = cheaper to fix                       |
| 4 | <b>Defect clustering</b>         | few modules hold most defects                  |
| 5 | <b>Pesticide paradox</b>         | repeated tests stop finding new bugs → revise  |
| 6 | <b>Context dependent</b>         | safety-critical ≠ e-commerce                   |
| 7 | <b>Absence-of-errors fallacy</b> | no bugs ≠ usable/useful product                |

- **4 Testing levels + who:** **Unit** → **Developer** · **Integration** → **Tester** · **System** → **Tester** · **Acceptance** → **End-user**.
- **Regression testing:** selects/prioritizes/executes existing tests — **designs none**.
- **Developer** = constructive, builds, driven by delivery · **Tester** = destructive, finds defects, driven by quality.

|          | <b>White-box (WBT)</b>                | <b>Black-box (BBT)</b>            |
|----------|---------------------------------------|-----------------------------------|
| Basis    | <b>Implementation / structural</b>    | <b>Specification / functional</b> |
| Sees     | source code (control flow, data flow) | external interface only           |
| Objects  | small units                           | large systems                     |
| Timeline | early (unit/component)                | late (system/acceptance)          |
| Tester   | <b>developers</b>                     | professional / 3rd-party (IV&V)   |

- **Gray-box** = mixed (middle levels, e.g. procedures in modules).
- **When to stop (Exit criteria):** *not* "no more defects found". **Resource-based** (time/money — irresponsible) vs **Quality-based** (goals reached) vs Activity completion.
- **Manual** = oldest, rigorous, hard-to-repeat, costly, slow · **Automated** = fast, repeatable, reliable, reusable, programmable.
- **Automate:** every-build/regression, data-driven, internal-detail (GUI), stress/load.
- **DON'T automate:** usability, logical errors, specs/design docs, one-time/ASAP, ad-hoc, tests without predictable results. **Automation cannot replace manual testing.**

# 01 — CHAPTER NOTES (full, in slide order)

---

Everything below comes straight from the slide decks, kept in lecture order. Short blocks; examples sit on their own lines.

**Which file is which chapter** (names are shifted +1): Ch.02 = Ch 1 · Ch.03 = Ch 2 · Ch.04 = Ch 3 · Ch.05 = Ch 4 · Ch.06 Software Testing = Ch 5 · Ch.06 Unit Testing = Ch 6.

## CHAPTER 1 — Basics of Software Quality Assurance

---

### What is software?

Software is **more than a computer program** — it is programs, procedures, and possibly associated documentation and data. Two major types: - **Generic (Buy)** — stand-alone, sold on the open market → **COTS** (commercial off-the-shelf). - **Customized (Build)** — built for a specific customer/business (bespoke). *Example:* a COTS office suite is used by thousands and may not do exactly what you want; bespoke software is built to one business's exact requirements.

### What is software quality?

- **ISO/IEC 9126:** the totality of functionality and features that contribute to a product's ability to satisfy **stated or implied needs**.
- **IEEE Std 610:** the degree to which a component/system/process meets specified requirements and/or user needs and expectations.

### Challenges in software projects

The primary goal of project management: deliver **on time, on budget, at expected quality**. Three main challenges: **Time** (late), **Cost** (over budget), **Scope** (low quality/faults).

### Examples of software defects

- **Therac-25** — radiation therapy machine by AECL (1986); people died from massive radiation overdose caused by a **software bug**.
- **Ariane 5** — 4 June 1996; the rocket tore itself apart **37 seconds** after launch due to a control-software malfunction — the **most expensive computer bug in history**.

### Software testing

Testing = executing a system/component under specified conditions to **find defects/bugs** and verify it satisfies requirements. Goals — detect issues in: - Requirement conformance - Errors in system operation - System performance (*example: TCAS in an autopilot system*) Levels: **Unit** (each module), **Integration** (combine modules, check incompatibility), **System** (software + hardware combined).

### Role of testing — two categories

- **Static analysis (manual testing):** examines the code/document and reasons over **all** behaviours that might arise at run time. *Examples:* code review, inspection, algorithm analysis. *Limitation:* review takes a long time.
- **Dynamic analysis (automated testing):** actual **program execution** to expose failures; observe representative behaviour, then conclude about quality. *Limitation:* cannot "think out of the box"; finds only syntax errors. Static and dynamic are **complementary** — combine the strengths of both.

### Software Quality Assurance (SQA)

The function that **assures** standards, processes and procedures are appropriate and correctly implemented — a **planned and systematic** approach. - An **umbrella activity** applied throughout the software process. - Monitors the engineering processes/methods to ensure quality. - *Role:* develop an effective **plan** for software development.

## Software Quality Control (SQC)

The function that **checks** the project follows its standards/processes/procedures and produces the required internal & external (deliverable) products. - Role: **execute** the plan effectively. - Activities: **Requirement Review, Design Review, Code Review, Deployment Plan Review, Test Plan Review, Test Cases Review**.

## Difference between SQA & SQC

- SQA gives **assurance** quality will be achieved; SQC **fulfils** the quality request.
- SQA aims to **prevent** defects; SQC aims to **identify and resolve** defects.
- SQA is the technique of **managing** quality; SQC is the method to **verify** quality.
- SQA **does not execute** the program; SQC **always executes** the program.
- SQA = **all team members** responsible; SQC = **only the testing team**.
- SQA = **Verification** (doing things right, process-oriented); SQC = **Validation** (results as expected, product-oriented).
- SQA = planning; SQC = action/execution.
- SQA = full **SDLC**; SQC = software **testing life cycle**.
- SQA = **failure prevention** system; SQC = **failure detection** system.

## Difficulties in achieving good quality

- **Size** — many LOC; manual testing struggles to find logical errors in bulky code.
- **Complexity** — software deals with logic; a logic error can have big impact (*example: autopilot "set landing mode"*).
- **Environmental stress/constraints** — test at maximum load (*example: result-publish website*).
- **Flexibility/adaptability** — frequent change (*example: regression testing after each change*).
- **Cost and market conditions** — testing cost of automated tools and licenses.

## Software Quality Engineering (SqE)

Ensures quality through **validation and verification** activities. Three stages: - **Planning** — quality planning. - **Execution** — execute the chosen QA plan / V&V activities. - **Decision** — decide pass/fail criteria of a test case; measurement & analysis to demonstrate quality.

## SqE activities

- **Testing** — remove defects and ensure quality.
- Other QA alternatives to testing:
- **Defect Prevention** (plan to avoid defects)
- **Formal Verification** (Inspection / Review / Walkthrough)
- **Fault Tolerance** (*examples: data backup, auto-save document, reliability up to 5 failures*)

## Error, Fault, Failure, Defect

- **Error** — a human action that produces an incorrect result (missing/incorrect human action that injects a fault).
- **Fault** — an incorrect step, process, or data definition in a program (underlying condition that causes failures).
- **Failure** — the inability of a system/component to perform its required function within specified performance requirements.
- **Defect** — a behavioural deviation from the user requirement or product specification. Failures, faults, and errors are collectively called **defects** — also commonly called "**bugs**".

## Complete testing

Objectives: detect bugs, reduce risk of defects, reduce cost of testing. **Complete/Exhaustive testing** = no undisclosed faults at end of test phase — **nearly impossible** because: - the input domain (valid + invalid) is too

large; - the design may be too complex, and budget too high; - you cannot create all possible execution environments (*example: every autopilot simulation*).

## Testing activities

Identify the objective → select inputs from the input domain → compute expected outcome → set up execution environment → execute → analyse results vs expected. A test exercises a **subset** of the input domain and a subset of program behaviour (divide domain D into D1, D2).

---

## CHAPTER 2 — Software Quality

---

### Quality perspectives

- **Subjects (people's perspectives):** **External/Consumer** (customers, users) · **Internal/Producer** (developers, testers, managers) · **Other** (3rd party, indirect users, e.g. email notification).
- **Objects:** software products, systems, and services.
- **External view** = mostly a **black box** (observe behaviour, can't see inside).
- **Internal view** = a **white/clear box** (see what is inside and how it works).

### Five views of quality (Kitchenham & Pfleeger)

1. **Mystical view (misconception)** — quality is recognized through experience but not definable; a good-quality object "stands out" and is easily recognized.
2. **User view** — quality = extent a product meets user needs/expectations; good if it satisfies many users; covers usability, reliability, efficiency.
3. **Manufacturing view** — **conformance** to process standards/requirements; deviation reduces quality; "**right the first time**" to cut cost; improve product by improving process. **CMM and ISO 9001** are based on this view.
4. **Product view** — good **internal** properties lead to good **external** properties.
5. **Value-based view** — customers' **willingness to pay**; merges **excellence** (a measure of quality) and **worth** (value); trades off cost vs quality.

### Why measure quality?

Measurement gives a **quantitative** view. Reasons: - **Baseline** — establish quality baselines (*example: extract all info from a website within 20 minutes*). - **Quality improvement based on cost** — each improvement has a cost; measurement drives process improvement. - **Know the present level** — for future planning; investigate improvement needs after measuring.

### Software Quality Factor

A quality **factor** = a **behavioural characteristic** of a system. *Examples:* Correctness, Reliability, Efficiency, Performance. Model used: **McCall's** Quality Factors and Criteria. - **Integrity/Security** — extent unauthorized access to software/data can be controlled (*example: only Auditor-privileged users can view transaction histories*). - **Efficiency** — computing resources and code required to perform a function (*example: ≥25% of processor/RAM unused at peak load*). - **Usability** — effort to learn, operate, prepare input, interpret output (*example: submit a request in avg 4 / max 6 minutes*). - **Reliability & Correctness** — extent a program performs intended functions with required precision; **reliability = probability of executing without failure for a specific period of time** (*example: ≤5 of 1000 runs lost to failures*).

### Software Quality Criteria

A quality **criterion** = an **attribute of a quality factor** related to software development. *Examples:* Modularity, Testability, Maintainability, Reusability. - **Modularity** — architecture attribute; cohesive components in one module → higher maintainability. - **Maintainability** — effort to locate and fix a defect in an operational program. - **Testability** — effort to ensure a program performs its intended functions (*example: max cyclomatic complexity of a module ≤ 20*). - **Flexibility** — effort to modify an operational program. - **Portability** — effort to transfer a program

from one hardware/software environment to another. - **Reusability** — extent parts of a system can be reused in other applications. - **Interoperability** — effort to couple one system with another (*examples: biometric SIM registration, blockchain, importing chemical structures from another tool*).

## ISO-9126 Quality Framework

- The **most influential** framework in software engineering today.
- A **hierarchical** framework: **quality characteristics** and **sub-characteristics**.
- **Six top-level characteristics**, each with its own **non-overlapping (exclusive) sub-characteristics**: 1. **Functionality** — what is needed 2. **Reliability** — functions correctly 3. **Usability** — effort to use 4. **Efficiency** — resources needed 5. **Maintainability** — correct/improve/adapt 6. **Portability** — from one environment to another

## Quality expectations

- **External/Consumer** — "good enough" for the **PRICE: Fit-for-use** (doing the right things) + **Conformance** (doing the things right). Expectations differ: General (functionality & reliability), Usability (GUI/web), Safety (safety-critical systems, e.g. autopilot).
- **Internal/Producer** — "good enough" for the **COST**: mirrors the consumer side; functionality & correctness via V&V; maintainability (service); interoperability (interfacing units); modularity (3rd party/outsourcing).

---

# CHAPTER 3 — Maturity Models

---

## Software process

A **process** = a set of activities executed to develop products (methods, techniques, strategies, procedures, practices) relying on repositories (documents, standards, policies). Benefits of a defined process: it can be **repeated**, **evaluated** (via metrics like cost, quality, time), and **improved**. Process tasks: Requirements Analysis, Design/Modeling, Coding, Testing, Implementation/Integration, Operation/Maintenance, Documentation.

## Process improvement (Verification)

To improve a defined process (from a **baseline** = existing practices), evaluate capabilities and limitations. Why improve a dev/test process: - **Quality** — better insight into quality characteristics. - **Lead Time** — saves testing time. - **Cost** — lower cost. Three models: - **CMM** — evaluate software **development** processes; supports incremental improvement. - **TPI** — improve the **testing** process. - **TMM** — evaluate a **testing** process.

## Capability Maturity Model (CMM)

An organization's **maturity level** tells to what extent it can produce **low-cost, high-quality** software. Knowing the current level, it can target the next. **Five levels**: 1. **Initial** — processes disorganized/chaotic; success depends on individual efforts; not repeatable (not sufficiently defined/documented). 2. **Repeatable** — basic project-management techniques established; successes can be repeated. 3. **Defined** — organization has its own standard process (documentation, standardization, integration). 4. **Managed** — organization monitors and controls its processes through **data collection and analysis**. 5. **Optimizing** — processes constantly improved through **feedback** and **innovative** processes.

## Test Process Improvement (TPI)

A test process = a way of performing activities related to **defect detection**, e.g. identifying test goals, preparing a test plan, identifying kinds of tests, hiring test personnel, designing test cases, procuring tools, assigning cases to engineers, prioritizing, organizing execution into cycles, executing, reporting defects. **How to improve a test process (4 steps)**: 1. Determine an area for improvement. 2. Evaluate the current state (baseline — quality, time, cost). 3. Identify the next desired state and the means to achieve it. 4. Implement the necessary changes.

## Testing Maturity Model (TMM)

Like development-process improvement, testing processes also need a framework to assess and improve; evaluation is key. - **Pioneered by Ilene Burnstein**. - Describes an evolutionary path of test-process maturity in **five levels**. - Each level = **Maturity goals**, **Supporting maturity goals**, and **ATRs** (Activities, Tasks, Responsibilities — views from manager, developer, tester, customer).

**TMM levels:** 1. **Initial** — no maturity goals; testing begins **after** code is written; done to show the system works; ad-hoc; progress not tracked; testing not seen as critical. 2. **Phase Definition** — develop testing & debugging goals; initiate a **test planning** process (identify objectives, analyze risks, devise strategies, develop specs, allocate resources); institutionalize basic testing techniques. 3. **Integration** — establish a **software test group**; establish a technical training program; integrate testing into the software lifecycle; control and monitor the testing process. 4. **Management and Measurement** — establish an org-wide review program; establish a test management program; evaluate software quality. 5. **Optimization / Defect Prevention and Quality Control** — apply process data for **defect prevention**; **statistical quality control**; test process **optimization**.

---

## CHAPTER 4 — Software Quality Assurance

---

### Quality Assurance

QA focuses on the **correctness** aspect of quality and dealing with defects. Three generic categories: - **Defect Prevention** — prevent certain faults from being **injected** (errors = missing/incorrect human actions) (*examples: default values, options to select*). - **Defect Reduction** — detect and remove faults once they've been injected. - **Defect Containment** — control defects via fault tolerance, failure prevention, or failure-impact minimization to assure reliability & safety (*example: auto-save*).

### Defect prevention

Two generic ways: - **Error source removal** — eliminate error sources (ambiguities, human misconceptions) via **education and training** (people-based solution). - **Error blocking** — directly correct/block the missing or incorrect human actions (*examples: user can't set the divisor to 0; data validation in Excel*) — a direct intervention that prevents fault injection. Also: **systematic defect prevention via process improvements** — formal methods prevent deviations from specs/design.

### Defect reduction

Prevention is **not 100% effective**, so reduce faults with: - **Inspection** — the most common **static** technique: critical reading/analysis of code, designs, specs, test plans. - **Informal reviews/walkthroughs** — self-conducted, independent, pass-around, "canteen discussion". - **Formal inspections** — multiple human inspectors, predefined coordination. - **Testing** — execute the software, observe behaviour; on failure, analyze to locate & fix the fault. - **Other / risk identification** — formal model-based analyses, **boundary value analysis**, **control-flow & data-flow analyses**, **simulation & prototyping** (*example: autopilot*).

### Defect containment

Reduction only lowers faults to a low level, not zero (large size, high complexity, complete testing impossible). Containment focuses on **failures** — contain them to local areas (no global failure) or limit damage (*example: exception handling*). Two generic ways: - **Software fault-tolerance** — **Recovery** (rollback and redo) · **NVP** (N-Version Programming — fault blocked). - **Safety assurance and failure containment**.

### Safety assurance & failure containment

- **Safety** — accident-free (*example: autopilot safety eliminates pilot error*).
- **Accident** — a failure with severe consequences.
- **Hazard** — a pre-condition to an accident.
- **Safety assurance** — hazard analysis, hazard elimination/reduction/control, damage control.

## QA in the software process

- **Mega-process:** Initiation → Development → Maintenance → Termination.
- **Development components:** Requirement, Specification, Design, Coding, Testing, Release.
- **Process variations:** Waterfall · Iterative (QA in increments) · Spiral (QA + risk management) · Agile (XP — test-driven development, pair programming) · Maintenance (focus on defect handling).

## Mapping DC view → V&V view

Defect-Centered (DC) activity → Verification/Validation focus: - **Defect prevention** → Both, mostly indirectly (Requirement-related = Validation; Project Plan = Verification; Formal specification = Validation). - **Defect reduction (testing)** → Both, mostly **Verification**. Unit = **Verification**; Integration = both (more verification); System = both; Acceptance = both (more validation); **Beta = Validation**. - **Defect containment** → Both, mostly **Validation**. Operation = Validation; Design & implementation = both (more verification).

---

# CHAPTER 5 — Software Quality Engineering (SQE)

---

## What SQE is

Meet or exceed quality expectations through the selection and execution of appropriate QA activities while **minimizing cost and project risks** under project constraints. It is an integral part of the overall software engineering process (cost and schedule are also managed).

## Generic testing process

- **Pre-QA activities — Quality/Test Planning:** most key decisions are made here. Set specific quality goals:
- Identify quality perspective & expectation (meaningful to target customers/users).
- Select **direct quality measures** (quantified values for efficiency, reliability, usefulness).
- Assess quality expectations vs **cost**. Form an overall QA strategy (low-level, test preparation): select appropriate QA activities; choose quality measurements/models.
- **Test procedure preparation — Test cases (micro-level):** a **test case** is a collection of entities and related information that allows a test to be executed; includes test-case allocation and sequencing from **simple to complex**.

## Test plan vs Test case vs Test suite

- **Test plan** — a **high-level** document describing objectives, scope, approach, resources, schedule, and focus of testing activities.
- **Test case** — a **low-level** document describing an input/action/event and its expected results, to determine if a feature works correctly.
- **Test suite (macro-level)** — the collection of individual test cases run in a sequence until stopping criteria are met.
- Reuse of test cases from earlier versions = **regression testing**.
- All test cases should form one integrated suite regardless of origin. (*A test-case template includes: Test Case ID, Priority, Module, Title, Description, Precondition, Dependencies, Test Steps, Test Data, Expected Results, Actual Results, Status Pass/Fail, Post Condition.*)

## Test execution & post-QA

- **In-QA — Test Execution:** execute planned QA activities and handle defects; collect failure info (what/where/when/severity); document activities (templates).
- **Post-QA — Quality Measurement, Assessment & Improvement:** follow-up, feedback, identify improvement potential. "**Post-QA**" ≠ **after QA finishes** — measurement/analysis runs **parallel** to QA; pre-QA may overlap QA too.

## Testing teams — organization & management

- Customers/users may act as **informal** testers (usability/beta).
- Independent professional testing organizations act as a **trusted intermediary** between vendors and customers.
- Team structures:
- **Vertical model** — organized **around a product**; dedicated people perform one or more testing tasks for that product.
- **Horizontal model** — performs **one kind of testing** for many different products.
- **Mixed model** — combines both; used by large software organizations.

---

## CHAPTER 6 — Software Testing

---

### Testing basics

Testing = execute the software and observe its behaviour/outcome. On failure, analyze the record to locate and fix the fault; otherwise gain confidence the software will do its job. Testing is a **distinct process**: reveals defects, shows quality attributes (reliability, performance); begins when the project is conceptualized; done by different people at different stages; uses many strategies/metrics; shaped by organizational policy; combines **manual and automated** execution.

### Seven principles of testing

1. **Testing shows the presence of defects** but cannot prove their absence.
2. **Exhaustive testing is impossible** — use risk analysis, time/cost analysis, and priorities.
3. **Early testing** — defects found earlier are easier and cheaper to fix.
4. **Defect clustering** — a few modules contain most defects; defects are **not** evenly distributed.
5. **Pesticide paradox** — repeating the same tests stops finding new defects; review and revise tests.
6. **Testing is context dependent** — safety-critical software is tested differently from an e-commerce site.
7. **Absence-of-errors fallacy** — finding/fixing defects doesn't help if the system is unusable or doesn't meet user needs; no defects found  $\neq$  ready to ship.

### Testing levels (and who does each)

- **Unit testing** — *Developer*: test individual units (procedures, methods) in isolation.
- **Integration testing** — *Tester*: assemble modules into larger subsystems and test.
- **System testing** — *Tester*: test the whole system (functionality, load, etc.).
- **Acceptance testing** — *End-user*: verify the customer's expectations.
- **Regression testing** — **no new test cases designed**; existing tests are selected, prioritized, and executed to ensure nothing broke in the new version.

### Testing & debugging (roles)

- Test and re-test = **test** activities; debugging & correcting defects = **developer** activities.
- **Developer** — implements requirements, designs & programs; creating a product is success; perceptions **constructive**; software is known and driven by **delivery**.
- **Tester** — plans testing, designs test cases; only concerned with finding defects; finding errors is success; perceptions **destructive**; software is unknown and driven by **quality**.

### Testing key questions

- **WHY** — demonstrate proper behaviour/quality; defect-free development (detection & removal).
- **HOW** — techniques/process: **test planning & preparation** → **test execution** → **analysis & follow-up**.
- **VIEW** — Functional/external/**black-box**; Structural/internal/**white-box**; **Gray-box** (mixed).
- **EXIT** — Functional coverage vs usage-based (quality/reliability goals).

## Testing techniques

- **White-box (WBT)** — implementation-based / **structural**; examines source code focusing on **control flow** and **data flow**; done by developers. Derives test cases that: exercise all independent paths at least once; exercise all logical decisions true & false; execute all loops at their boundaries; exercise internal data structures.
- **Black-box (BBT)** — specification-based / **functional**; examines the program from outside at the external interface; apply input, observe outcome. The SQA group finds: incorrect/missing functions; interface errors; external database access errors; behaviour/performance errors; initialization/termination errors.
- **Levels of abstraction:** high-level whole system → **black-box** (late); low-level statements/data → **white-box** (early/small); middle levels → **gray-box** (*example: procedures individually as black box, procedure interconnection as white box at module level*).

## WBT vs BBT (detailed)

- **Perspective:** BBT = black box (input–output/external behaviour); WBT = glass box (internal implementation visible).
- **Objects:** WBT for small objects; BBT for large systems.
- **Timeline:** WBT early (unit/component); BBT late (system/acceptance).
- **Defect focus:** BBT observes external-function failures; WBT observes internal-implementation failures.
- **Detection & fixing:** WBT defects are easier to fix; WBT may **miss omission/design problems** (BBT catches these); BBT is effective for interface/interaction problems, WBT for problems localized in a small unit.
- **Techniques:** BBT if external functions are modeled; WBT if internal implementations are modeled.
- **Tester:** BBT by professional testers / third-party (IV&V); WBT by developers.

## When to stop testing (exit criteria)

- "Not finding defects anymore" is **NOT** an appropriate stopping criterion — user acceptance is what matters at the end.
- **Resource-based** — stop when time/money runs out (irresponsible for product quality).
- **Quality-based** — stop when quality goals (usability, reliability, real customer-usage resemblance) are reached.
- Substitute: **activity completion** — stop when planned test activities are done.

## Manual vs Automated testing

- **Manual** — oldest and most rigorous; a tester performs operations; hard to repeat, not always reliable, costly, time-consuming, labour-intensive.
- **Automated** — uses software tools; runs without manual intervention; fast, repeatable, reliable, reusable, programmable, saves time.

## Test automation

- **Key issues:** needs/potential for automation & tool selection; user training & time/effort; overall cost (acquisition, support, training, usage); impact on resources/schedule/management.
- **Pre-requisites:** system stable & well-defined; test cases unambiguous; tools & infrastructure in place; engineers have prior automation experience; adequate budget.
- **Automate:** tests run for every build (repetitive/**regression**); data-driven tests (multiple values); tests needing internal details (GUI attributes); **stress/load** testing.
- **Don't automate:** usability testing; logical errors; specification/design documents; one-time / "ASAP" testing; ad-hoc/random testing; tests without predictable results.
- **Automated testing CANNOT replace manual testing.**

## 02 — MOCK EXAM (questions only)

---

**SQAT (CSC 4133) — Mid · Chapters 1–6 Total: 100 marks** — Section A (40 × 1) · Section B (20 × 1) · Section C (5 × 8) Time yourself: aim for ~35 min on A+B, ~55 min on C. Answers are in `03_ANSWER_KEY.md` — don't peek until you've finished.

### SECTION A — Multiple Choice (40 × 1 = 40 marks)

---

*Choose the single best answer.*

1. Software is best defined as: a) only a computer program b) programs, procedures, documentation and data c) hardware and firmware d) source code only
2. COTS (commercial off-the-shelf) software is an example of which type? a) Customized b) Bespoke c) Generic d) Embedded
3. Which statement correctly describes SQA? a) It executes the program and detects defects b) It aims to prevent defects and does not execute the program c) Only the testing team is responsible d) It is a failure-detection system
4. The Ariane 5 rocket disaster was caused by: a) a hardware failure b) pilot error c) a control-software malfunction d) a fuel leak
5. A human action that produces an incorrect result is a(n): a) Fault b) Failure c) Error d) Defect
6. "The inability of a system or component to perform its required function" defines a: a) fault b) failure c) error d) hazard
7. Which type of analysis actually executes the program? a) Static b) Dynamic c) Inspection d) Code review
8. SQA is described as a(n) \_\_\_\_\_ activity applied throughout the software process. a) umbrella b) one-time c) post-release d) optional
9. The five views of software quality were proposed by: a) McCall b) Kitchenham & Pfleeger c) Ilene Burnstein d) ISO
10. Which view holds that quality is recognized through experience but cannot be defined? a) User b) Product c) Mystical d) Value-based
11. The CMM and ISO 9001 models are based on which view of quality? a) User b) Manufacturing c) Product d) Mystical
12. Reliability is an example of a quality: a) criterion b) factor c) metric d) standard
13. Modularity is an example of a quality: a) factor b) criterion c) view d) model
14. Which of the following is **NOT** one of the six top-level ISO-9126 characteristics? a) Functionality b) Portability c) Security d) Usability
15. "The probability of software executing without failure for a specific period of time" defines: a) Efficiency b) Usability c) Reliability d) Portability
16. The value-based view of quality merges which two concepts? a) cost and schedule b) excellence and worth c) usability and reliability d) internal and external
17. How many maturity levels does the CMM have? a) 3 b) 4 c) 5 d) 6
18. CMM Level 4 is: a) Defined b) Managed c) Optimizing d) Repeatable
19. CMM Level 5 is: a) Managed b) Optimizing c) Defined d) Initial
20. The Testing Maturity Model (TMM) was pioneered by: a) McCall b) Kitchenham c) Ilene Burnstein d) Jeff Tian
21. At which CMM level are processes "disorganized, even chaotic"? a) Initial b) Repeatable c) Defined d) Managed
22. Which model is used to **evaluate a testing** process? a) CMM b) TMM c) ISO 9001 d) McCall
23. TMM Level 3 is: a) Phase Definition b) Integration c) Management and Measurement d) Optimization
24. The three generic QA categories are prevention, reduction, and: a) detection b) containment c) correction d) validation

25. The auto-save feature of a document editor is an example of defect: a) prevention b) reduction c) containment d) removal
26. N-Version Programming (NVP) is a form of: a) defect prevention b) fault tolerance / containment c) inspection d) validation
27. A "pre-condition to an accident" is called a: a) hazard b) failure c) fault d) error
28. Which is the most commonly used **static** technique to reduce defects? a) Testing b) Simulation c) Inspection d) Prototyping
29. Which practices are highlighted for the Agile/XP process? a) waterfall stages b) test-driven development & pair programming c) big design up front d) hazard analysis
30. A test plan is a \_\_ **document, whereas a test case is a** \_\_\_\_ document. a) low-level; high-level b) high-level; low-level c) macro; macro d) internal; external
31. A collection of individual test cases run in sequence until stopping criteria are met is a: a) test plan b) test case c) test suite d) test cycle
32. Reusing test cases from earlier versions of the software is called: a) unit testing b) regression testing c) smoke testing d) acceptance testing
33. A testing team organized around one product, with dedicated people, uses the \_\_\_\_\_ model. a) horizontal b) vertical c) mixed d) matrix
34. The horizontal team model: a) does all testing for one product b) performs one kind of testing for many products c) does no testing d) does only unit testing
35. In SQE, most of the key decisions about testing are made during: a) test execution b) pre-QA / planning c) post-QA d) maintenance
36. Which principle states that testing can show defects exist but cannot prove they are absent? a) Principle 1 b) Principle 2 c) Principle 5 d) Principle 7
37. The "pesticide paradox" means: a) tests are toxic b) repeating the same tests eventually finds no new defects c) defects cluster d) testing is context dependent
38. Unit testing is typically performed by the: a) end-user b) tester c) developer d) manager
39. White-box testing is also known as: a) functional testing b) structural testing c) acceptance testing d) beta testing
40. Which of the following **cannot** be fully automated? a) regression testing b) load testing c) usability testing d) data-driven testing
- 

## SECTION B — True / False (20 × 1 = 20 marks)

---

Write T or F.

1. Software Quality Control (SQC) always involves executing the program. 2. In SQA, only the testing team is responsible for quality. 3. Static analysis executes the program to expose failures. 4. A fault is a human action that produces an incorrect result. 5. Security is one of the six top-level ISO-9126 quality characteristics. 6. The manufacturing view of quality is based on conformance to requirements. 7. CMM Level 5 is "Managed". 8. The TMM has five maturity levels. 9. Defect-prevention activities can be 100% effective. 10. N-Version Programming (NVP) is a defect-containment technique. 11. A test case is a high-level document. 12. Regression testing designs brand-new test cases. 13. Testing shows the presence of defects but cannot prove their absence. 14. Exhaustive testing is practical for most systems. 15. Acceptance testing is performed by the end-user. 16. White-box testing is performed mainly by developers. 17. Automated testing can fully replace manual testing. 18. Reliability is a quality criterion, not a quality factor. 19. The five views of software quality were proposed by Kitchenham and Pfleeger. 20. A hazard is a failure with severe consequences.

---

## SECTION C — Theory (5 × 8 = 40 marks)

---

Answer all five. Each answer must include a definition, a structure/list, and at least one example.

- C1.** Differentiate between **SQA** and **SQC**. Give at least **five** points of difference, and state which one maps to verification and which to validation. (8)
- C2.** Describe the **five views of software quality** proposed by **Kitchenham and Pfleeger**, giving one example or note for each view. (8)
- C3.** List and explain the **five levels of the CMM**. Then briefly explain how **TMM** and **TPI** differ from CMM in purpose. (8)
- C4.** Explain the **three generic categories of QA activities** (defect prevention, defect reduction, defect containment) with an example of each. For defect prevention and defect containment, also name their **two generic techniques**. (8)
- C5.** (a) Distinguish between a **test plan**, a **test case**, and a **test suite**. (b) Compare **white-box** and **black-box** testing on at least **four** dimensions, with an example of each. (8)

## 03 — ANSWER KEY

---

### SECTION A — MCQ answers

| Q  | Ans      | Q  | Ans      | Q  | Ans      | Q  | Ans      |
|----|----------|----|----------|----|----------|----|----------|
| 1  | <b>B</b> | 11 | <b>B</b> | 21 | <b>A</b> | 31 | <b>C</b> |
| 2  | <b>C</b> | 12 | <b>B</b> | 22 | <b>B</b> | 32 | <b>B</b> |
| 3  | <b>B</b> | 13 | <b>B</b> | 23 | <b>B</b> | 33 | <b>B</b> |
| 4  | <b>C</b> | 14 | <b>C</b> | 24 | <b>B</b> | 34 | <b>B</b> |
| 5  | <b>C</b> | 15 | <b>C</b> | 25 | <b>C</b> | 35 | <b>B</b> |
| 6  | <b>B</b> | 16 | <b>B</b> | 26 | <b>B</b> | 36 | <b>A</b> |
| 7  | <b>B</b> | 17 | <b>C</b> | 27 | <b>A</b> | 37 | <b>B</b> |
| 8  | <b>A</b> | 18 | <b>B</b> | 28 | <b>C</b> | 38 | <b>C</b> |
| 9  | <b>B</b> | 19 | <b>B</b> | 29 | <b>B</b> | 39 | <b>B</b> |
| 10 | <b>C</b> | 20 | <b>C</b> | 30 | <b>B</b> | 40 | <b>C</b> |

**Justifications for the tricky ones:** - **2 – C (Generic):** COTS = sold on the open market to many users; "Buy", not "Build". - **3 – B:** SQA prevents defects and does **not** execute the program; executing/detecting is SQC. - **5 vs 6:** *Error* = human action; *Failure* = inability to perform the function. Don't swap them. - **12 / 13:** Reliability = **factor** (behavioural characteristic); Modularity = **criterion** (attribute of a factor). - **14 – C:** Security is **not** an ISO-9126 top-level characteristic (the six are Functionality, Reliability, Usability, Efficiency, Maintainability, Portability). - **18 / 19:** Level 4 = **Managed**, Level 5 = **Optimizing** — a classic reversal trap. - **25 / 26:** Auto-save and NVP are **containment**, not prevention. - **30 – B:** Test plan = high-level; test case = low-level. - **32 – B:** Regression **reuses** existing tests; it does not design new ones. - **36 – A:** Principle 1 = presence, not absence. - **40 – C:** Usability testing cannot be fully automated.

---

## SECTION B — True/False answers

| Q  | Ans      | Why (if tricky)                                                                |
|----|----------|--------------------------------------------------------------------------------|
| 1  | <b>T</b> | SQC always executes the program.                                               |
| 2  | <b>F</b> | In SQA <b>all team members</b> are responsible; "only testing team" is SQC.    |
| 3  | <b>F</b> | <b>Dynamic</b> analysis executes the program; static does not.                 |
| 4  | <b>F</b> | That defines an <b>error</b> , not a fault.                                    |
| 5  | <b>F</b> | Security is <b>not</b> an ISO-9126 top-level characteristic.                   |
| 6  | <b>T</b> | Manufacturing view = conformance to requirements.                              |
| 7  | <b>F</b> | Level 5 = <b>Optimizing</b> ; Level 4 = Managed.                               |
| 8  | <b>T</b> | TMM has 5 levels.                                                              |
| 9  | <b>F</b> | Prevention is <b>never</b> 100% effective.                                     |
| 10 | <b>T</b> | NVP is a fault-tolerance / containment technique.                              |
| 11 | <b>F</b> | Test case is <b>low-level</b> .                                                |
| 12 | <b>F</b> | Regression <b>reuses</b> tests; designs none.                                  |
| 13 | <b>T</b> | Principle 1.                                                                   |
| 14 | <b>F</b> | Exhaustive testing is nearly impossible.                                       |
| 15 | <b>T</b> | Acceptance testing = end-user.                                                 |
| 16 | <b>T</b> | WBT is done mainly by developers.                                              |
| 17 | <b>F</b> | Automation <b>cannot</b> replace manual testing.                               |
| 18 | <b>F</b> | Reliability is a <b>factor</b> , not a criterion.                              |
| 19 | <b>T</b> | Kitchenham & Pfleeger.                                                         |
| 20 | <b>F</b> | That's an <b>accident</b> ; a hazard is a <b>pre-condition</b> to an accident. |

## SECTION C — Model answers

(Write them in this shape: definition → list/structure → example. Mark split shown in brackets.)

### C1 — SQA vs SQC (8)

**Definition [2]: SQA (Software Quality Assurance)** is a planned, systematic, process-oriented set of activities that assures the standards, processes and procedures are appropriate and correctly implemented — an umbrella activity across the whole SDLC. **SQC (Software Quality Control)** is a product-oriented set of activities that checks the project follows those standards and produces the required deliverables, by executing/evaluating the work product.

**Differences [5] (any five):**

| SQA                                                | SQC                                                        |
|----------------------------------------------------|------------------------------------------------------------|
| Aim: <b>prevent</b> defects                        | Aim: <b>identify &amp; resolve</b> defects                 |
| <b>Does not execute</b> the program                | <b>Always executes</b> the program                         |
| <b>All team members</b> responsible                | <b>Only the testing team</b> responsible                   |
| Maps to <b>Verification</b> (doing things right)   | Maps to <b>Validation</b> (results as expected)            |
| Failure <b>prevention</b> system; full <b>SDLC</b> | Failure <b>detection</b> system; <b>testing life cycle</b> |

**Verification/Validation [1]:** SQA = **Verification**; SQC = **Validation**.

**Example:** Defining the coding standard and review checklist for a login module = **SQA**; actually running the login test cases and logging the bugs = **SQC**.

---

## C2 — Five views of quality (Kitchenham & Pfleeger) (8)

**Definition [1]:** Kitchenham and Pfleeger describe quality from five different perspectives because different stakeholders judge quality differently.

**The five views [5] + examples [2] (≈1.5 each):** 1. **Mystical view** — quality is recognized through experience but cannot be defined; a good product "stands out". *Example:* "I know good software when I see it." 2. **User view** — quality = the extent to which the product meets user needs/expectations; good if it satisfies many users (usability, reliability, efficiency). *Example:* an app rated highly by most users. 3. **Manufacturing view** — conformance to process standards/requirements; "right the first time"; improve the product by improving the process. *Example:* CMM/ISO 9001-based development. 4. **Product view** — good **internal** properties produce good **external** properties. *Example:* clean modular code → fewer field failures. 5. **Value-based view** — customers' willingness to pay; merges **excellence** (quality) and **worth** (value); trades cost off against quality. *Example:* a "good enough for the price" product.

*(Mnemonic: My Uncle Makes Pretty Vases.)*

---

## C3 — CMM five levels + how TMM/TPI differ (8)

**Definition [1]:** The CMM measures how mature an organization's software **development** process is — i.e. to what extent it can produce low-cost, high-quality software.

**Five levels [5]:** 1. **Initial** — processes disorganized/chaotic; success depends on individuals; not repeatable. 2. **Repeatable** — basic project management established; successes can be repeated. 3. **Defined** — the organization has its own standard process (documentation, standardization, integration). 4. **Managed** — processes monitored and controlled via **data collection and analysis**. 5. **Optimizing** — processes continuously improved via feedback and innovation.

*(Mnemonic: Initial, Repeatable, Defined, Managed, Optimizing.)*

**How TMM/TPI differ [2]:** - **TMM** (pioneered by **Ilene Burnstein**) also has 5 levels but **evaluates/improves the testing process**, not development. - **TPI** is a model specifically for **improving** the testing process (4 steps: determine area → evaluate current state → identify next state → implement changes).

**Example:** A company that only tests after coding, ad hoc, with no tracking, sits at **CMM Level 1 / TMM Level 1**.

---

## C4 — Three QA categories (8)

**Definition [1]:** QA focuses on the correctness aspect of quality; its activities fall into three generic categories that deal with defects at different points.

**The three categories [4] + examples [1]:** 1. **Defect Prevention** — stop faults from being **injected** in the first place. *Example:* providing default values / valid options so wrong input can't be entered. 2. **Defect Reduction** — **detect and remove** faults that have already been injected. *Example:* inspection and testing. 3. **Defect Containment** — **contain failures** to local areas or **limit the damage** to assure reliability/safety. *Example:* auto-save / exception handling.

**Two techniques each [2]:** - **Defect prevention:** ① **error source removal** (education & training — people-based) ② **error blocking** (e.g. don't allow a 0 divisor; data validation). - **Defect containment:** ① **software fault-tolerance** (Recovery = rollback & redo; **NVP** = N-Version Programming) ② **safety assurance & failure containment** (hazard analysis, damage control).

*(Note: prevention is never 100% effective — that's why reduction and containment exist.)*

---

## C5 — Test plan/case/suite + WBT vs BBT (8)

**(a) Definitions [3]:** - **Test plan** — a **high-level** document describing the objectives, scope, approach, resources, schedule and focus of testing. - **Test case** — a **low-level** document describing one input/action/event and its expected result, to check if a feature works. - **Test suite** — the **macro-level** collection of individual test cases run in sequence until stopping criteria are met (reusing earlier cases = regression testing).

### **(b) White-box vs Black-box [4] + examples [1]:**

| Dimension   | White-box (structural)       | Black-box (functional)             |
|-------------|------------------------------|------------------------------------|
| Basis       | implementation / source code | specification / external interface |
| Focus       | control flow & data flow     | input–output behaviour             |
| Object size | small units                  | large systems                      |
| Timeline    | early (unit/component)       | late (system/acceptance)           |
| Tester      | developers                   | professional / third-party (IV&V)  |

*Example:* Checking every branch of a function's `if/else` = **white-box**; entering a username/password on the login screen and checking the result without seeing the code = **black-box**. (*Middle levels = gray-box.*)

## 04 — RAPID FIRE (100 Q&A)

---

Cover the question, say the answer, then check. Morning-before-exam drill.

### Chapter 1 — Basics of SQA

---

1. Two major types of software? — Generic (Buy) and Customized (Build).
2. What does COTS stand for? — Commercial off-the-shelf.
3. "Bespoke" software is which type? — Customized (Build).
4. Software is composed of what? — Programs, procedures, documentation, and data.
5. Software quality per ISO/IEC 9126? — Totality of functionality and features that satisfy stated or implied needs.
6. Software quality per IEEE Std 610? — Degree to which a system/component/process meets specified requirements and user needs.
7. Three main project challenges? — Time, Cost, Scope.
8. What was Therac-25? — A radiation-therapy machine (1986) that overdosed patients due to a software bug.
9. Which company built Therac-25? — AECL (Atomic Energy of Canada Ltd).
10. When did Ariane 5 fail, and how long after launch? — 4 June 1996, 37 seconds after launch.
11. Why is Ariane 5 famous? — The most expensive computer bug in history (control-software fault).
12. Definition of software testing? — Executing a system under specified conditions to find defects and verify requirements.
13. Two categories of quality assessment? — Static (manual) analysis and Dynamic (automated) analysis.
14. Which analysis executes the program? — Dynamic analysis.
15. Three examples of static analysis? — Code review, inspection, algorithm analysis.
16. SQA is what kind of activity? — An umbrella activity applied throughout the software process.
17. Does SQA execute the program? — No.
18. Does SQC execute the program? — Yes.
19. Who is responsible in SQA vs SQC? — SQA = all team members; SQC = only the testing team.
20. SQA and SQC map to which of Verification/Validation? — SQA = Verification; SQC = Validation.
21. Define Error / Fault / Failure / Defect (one line each)? — Error = human action producing wrong result; Fault = incorrect step/data in the program; Failure = system can't perform its function; Defect = deviation from spec.
22. Three stages of SqE? — Planning, Execution, Decision.

### Chapter 2 — Software Quality

---

23. External view sees software as? — A black box.
24. Internal view sees software as? — A white / clear box.
25. Who proposed the five views of quality? — Kitchenham and Pfleeger.
26. Name the five views of quality. — Mystical, User, Manufacturing, Product, Value-based.
27. Which view underlies CMM and ISO 9001? — Manufacturing view.
28. Which view says quality is recognized but not defined? — Mystical view.
29. The value-based view merges which two concepts? — Excellence (quality) and worth (value).
30. Product view claim? — Good internal properties lead to good external properties.
31. Three reasons to measure quality? — Baseline; quality improvement based on cost; know present level for planning.

- 32.** What is a quality factor? — A behavioural characteristic of a system.
- 33.** Examples of quality factors? — Correctness, reliability, efficiency, performance.
- 34.** What is a quality criterion? — An attribute of a quality factor, related to development.
- 35.** Examples of quality criteria? — Modularity, testability, maintainability, reusability.
- 36.** Whose model gives factors and criteria? — McCall's.
- 37.** Definition of reliability? — Probability of executing without failure for a specific period of time.
- 38.** Most influential quality framework? — ISO-9126.
- 39.** How many ISO-9126 top-level characteristics, and what are they? — Six: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability.
- 40.** Is security an ISO-9126 top-level characteristic? — No.

## Chapter 3 — Maturity Models

---

- 41.** What is a software process? — A set of activities executed to develop products.
- 42.** Three benefits of a defined process? — Repeatable, evaluable (via metrics), improvable.
- 43.** Three reasons to improve a process? — Quality, Lead Time, Cost.
- 44.** What does CMM evaluate? — Software development processes.
- 45.** What does TMM evaluate? — A testing process.
- 46.** What does TPI do? — Improves the testing process.
- 47.** How many CMM levels? — 5.
- 48.** Name the five CMM levels. — Initial, Repeatable, Defined, Managed, Optimizing.
- 49.** CMM Level 1? — Initial.
- 50.** CMM Level 4? — Managed.
- 51.** CMM Level 5? — Optimizing.
- 52.** Which CMM level uses data collection and analysis? — Level 4 (Managed).
- 53.** How many TMM levels? — 5.
- 54.** Who pioneered TMM? — Ilene Burnstein.
- 55.** Name the five TMM levels. — Initial, Phase Definition, Integration, Management and Measurement, Optimization.
- 56.** How many steps to improve a test process (TPI)? — 4.

## Chapter 4 — Software Quality Assurance

---

- 57.** Three generic QA categories? — Defect Prevention, Defect Reduction, Defect Containment.
- 58.** Two ways to do defect prevention? — Error source removal and error blocking.
- 59.** Error source removal is done via? — Education and training (people-based).
- 60.** Most common static defect-reduction technique? — Inspection.
- 61.** Two forms of inspection formality? — Informal walkthroughs/reviews and formal inspections.
- 62.** Two ways to do defect containment? — Software fault-tolerance and safety assurance.
- 63.** What is NVP? — N-Version Programming (a fault-tolerance technique).
- 64.** Recovery in fault tolerance means? — Rollback and redo.
- 65.** Define safety / accident / hazard. — Safety = accident-free; Accident = failure with severe consequences; Hazard = pre-condition to an accident.
- 66.** Is defect prevention 100% effective? — No.

- 67.** Name the four development process variations. — Waterfall, Iterative, Spiral, Agile (plus Maintenance).
- 68.** In the DC→V&V mapping, unit testing maps to? — Verification.

## Chapter 5 — Software Quality Engineering

---

- 69.** What does SQE minimize while meeting quality? — Cost and project risks.
- 70.** Three phases of the generic testing process? — Pre-QA, In-QA, Post-QA.
- 71.** When are most key testing decisions made? — Pre-QA / planning.
- 72.** Test plan is which level? — High-level.
- 73.** Test case is which level? — Low-level.
- 74.** What is a test suite? — A collection of test cases run until stopping criteria (macro-level).
- 75.** Reusing earlier test cases is called? — Regression testing.
- 76.** Does "Post-QA" mean after QA finishes? — No; it runs parallel to QA.
- 77.** Three testing team models? — Vertical, Horizontal, Mixed.
- 78.** The horizontal model does what? — One kind of testing for many different products.

## Chapter 6 — Software Testing

---

- 79.** How many principles of testing? — 7.
- 80.** Principle 1? — Testing shows presence of defects, not their absence.
- 81.** Principle 2? — Exhaustive testing is impossible.
- 82.** Principle 3? — Early testing (defects are cheaper to fix early).
- 83.** Principle 4? — Defect clustering.
- 84.** Principle 5? — Pesticide paradox.
- 85.** Principle 6? — Testing is context dependent.
- 86.** Principle 7? — Absence-of-errors fallacy.
- 87.** Four testing levels? — Unit, Integration, System, Acceptance.
- 88.** Who does unit testing? — Developer.
- 89.** Who does integration and system testing? — Tester.
- 90.** Who does acceptance testing? — End-user.
- 91.** Does regression testing design new tests? — No; it selects, prioritizes, and executes existing ones.
- 92.** White-box testing is also called? — Structural / implementation-based testing.
- 93.** Black-box testing is also called? — Functional / specification-based testing.
- 94.** Two things white-box examines? — Control flow and data flow.
- 95.** Who performs white-box testing? — Developers.
- 96.** Who performs black-box testing? — Professional / third-party testers (IV&V).
- 97.** What is gray-box testing? — Mixed black-box and white-box (middle levels).
- 98.** Is "no more defects found" a valid stopping criterion? — No.
- 99.** Two types of exit criteria? — Resource-based and quality-based.
- 100.** Can automation replace manual testing? — No.

## 05 — CONFUSION TABLE (the pairs students mix up)

Each block: the one-sentence difference, then the side-by-side table.

### 1. SQA vs SQC

**Difference:** SQA **prevents** defects by managing the process (never runs the program); SQC **finds** defects by evaluating the product (always runs the program).

| Aspect            | SQA                | SQC                        |
|-------------------|--------------------|----------------------------|
| Aim               | Prevent defects    | Identify & resolve defects |
| Runs the program? | <b>No</b>          | <b>Yes</b>                 |
| Orientation       | Process-oriented   | Product-oriented           |
| Maps to           | Verification       | Validation                 |
| Responsibility    | All team members   | Only testing team          |
| System type       | Failure prevention | Failure detection          |
| Scope             | Full SDLC          | Testing life cycle         |

### 2. Error vs Fault vs Failure vs Defect

**Difference:** an **error** (human) injects a **fault** (in the code), which when executed causes a **failure** (wrong behaviour); all three are collectively called **defects** ("bugs").

| Term           | What it is                                      | Where          |
|----------------|-------------------------------------------------|----------------|
| <b>Error</b>   | Human action producing an incorrect result      | In a person    |
| <b>Fault</b>   | Incorrect step/process/data definition          | In the program |
| <b>Failure</b> | Inability to perform the required function      | At run time    |
| <b>Defect</b>  | Behavioural deviation from spec (umbrella term) | Anywhere       |

### 3. Verification vs Validation

**Difference:** verification asks "are we building it **right**?" (process); validation asks "are we building the **right** thing?" (product).

| Verification           | Validation                    |
|------------------------|-------------------------------|
| Doing the things right | Results are what you expected |
| Process-oriented       | Product-oriented              |
| Linked to SQA          | Linked to SQC                 |
| Reviews, inspections   | Testing, user acceptance      |

### 4. Static vs Dynamic analysis

**Difference:** static analysis reads the code/documents **without running** it; dynamic analysis **executes** the program to see failures.

| Static (manual)                             | Dynamic (automated)                                 |
|---------------------------------------------|-----------------------------------------------------|
| No execution                                | Actual execution                                    |
| Reasons over all possible behaviours        | Observes representative behaviour                   |
| Code review, inspection, algorithm analysis | Running test cases                                  |
| Slow to review                              | Finds only what runs; "can't think outside the box" |

## 5. Quality Factor vs Quality Criterion

**Difference:** a **factor** is an external behavioural characteristic of the system; a **criterion** is an internal attribute of a factor tied to development.

| Quality Factor                                                          | Quality Criterion                                                               |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Behavioural characteristic of the system                                | Attribute of a quality factor                                                   |
| External view                                                           | Development-related                                                             |
| Reliability, Correctness, Efficiency, Performance, Usability, Integrity | Modularity, Testability, Maintainability, Reusability, Flexibility, Portability |
| From McCall's model                                                     | From McCall's model                                                             |

**Trap:** Reliability = factor; Modularity = criterion.

## 6. White-box vs Black-box vs Gray-box

**Difference:** white-box tests the **inside** (code), black-box tests the **outside** (behaviour), gray-box **mixes** both at the middle level.

|             | White-box                         | Black-box                        | Gray-box                       |
|-------------|-----------------------------------|----------------------------------|--------------------------------|
| Also called | Structural / implementation-based | Functional / specification-based | Mixed                          |
| Sees        | Source code (control & data flow) | External interface only          | Both, partially                |
| Object size | Small units                       | Large systems                    | Middle (procedures in modules) |
| Timeline    | Early (unit/component)            | Late (system/acceptance)         | Middle                         |
| Tester      | Developers                        | Professional / 3rd-party (IV&V)  | —                              |

## 7. CMM vs TMM (levels side by side)

**Difference:** CMM measures the **development** process maturity; TMM measures the **testing** process maturity (by Ilene Burnstein).

| Level | CMM (development) | TMM (testing)                         |
|-------|-------------------|---------------------------------------|
| 1     | Initial           | Initial                               |
| 2     | Repeatable        | Phase Definition                      |
| 3     | Defined           | Integration                           |
| 4     | <b>Managed</b>    | Management and Measurement            |
| 5     | <b>Optimizing</b> | Optimization / Defect Prevention & QC |

**Trap:** CMM 4 = Managed, 5 = Optimizing (don't reverse).

## 8. Test Plan vs Test Case vs Test Suite

**Difference:** the **plan** is the high-level strategy, a **case** is one low-level input+expected-result, and a **suite** is the whole set of cases run together.

|         | Test Plan                                        | Test Case                          | Test Suite                  |
|---------|--------------------------------------------------|------------------------------------|-----------------------------|
| Level   | High-level                                       | Low-level                          | Macro-level                 |
| Content | Objectives, scope, approach, resources, schedule | One input/action + expected result | Collection of test cases    |
| Purpose | How testing will be done                         | Check one feature                  | Run until stopping criteria |

## 9. Manual vs Automated testing

**Difference:** manual testing is human-run (rigorous but slow); automated testing is tool-run (fast but can't judge usability) — and automation **can't fully replace** manual.

| Manual                                                  | Automated                                                     |
|---------------------------------------------------------|---------------------------------------------------------------|
| Oldest, most rigorous                                   | Uses software tools                                           |
| Tester performs each step                               | Runs without manual intervention                              |
| Hard to repeat, costly, slow, labour-intensive          | Fast, repeatable, reliable, reusable, programmable            |
| Needed for: usability, logical errors, ad-hoc, one-time | Good for: regression, data-driven, load/stress, GUI internals |

## 10. Defect Prevention vs Reduction vs Containment

**Difference:** prevention stops faults being **injected**, reduction **removes** injected faults, containment **limits the damage** of the failures that remain.

|            | Prevention                           | Reduction                 | Containment                                       |
|------------|--------------------------------------|---------------------------|---------------------------------------------------|
| Acts on    | Errors (before injection)            | Faults (after injection)  | Failures (at run time)                            |
| Goal       | Don't inject the fault               | Detect & remove the fault | Contain/limit the failure                         |
| Techniques | Error source removal, error blocking | Inspection, testing       | Fault tolerance (NVP, recovery), safety assurance |
| Example    | Default values, data validation      | Code inspection           | Auto-save, exception handling                     |

**Trap:** NVP & auto-save = **containment**, not prevention. Prevention is never 100%.

## 11. Internal vs External view

**Difference:** the external/consumer view sees a **black box** (behaviour only); the internal/producer view sees a **white box** (the workings).

| External / Consumer                           | Internal / Producer                           |
|-----------------------------------------------|-----------------------------------------------|
| Customers and users                           | Developers, testers, managers                 |
| Black box                                     | White / clear box                             |
| "Good enough" for the <b>price</b>            | "Good enough" for the <b>cost</b>             |
| Functionality, reliability, usability, safety | Maintainability, interoperability, modularity |

## 12. Developer vs Tester role

**Difference:** the developer **builds** the product (constructive, driven by delivery); the tester tries to **break** it (destructive, driven by quality).

| Developer                                  | Tester                                 |
|--------------------------------------------|----------------------------------------|
| Implements requirements, designs, programs | Plans testing, designs test cases      |
| Success = creating a product               | Success = finding errors               |
| Perceptions constructive                   | Perceptions destructive                |
| Software is known; driven by delivery      | Software is unknown; driven by quality |
| Debugging & correcting defects             | Test & re-test                         |

## 13. Vertical vs Horizontal vs Mixed testing team

**Difference:** vertical = many testing tasks for **one product**; horizontal = **one testing type** across many products; mixed = both.

| Vertical                                           | Horizontal                            | Mixed                       |
|----------------------------------------------------|---------------------------------------|-----------------------------|
| Organized around one product                       | Organized around one testing type     | Combines both               |
| Dedicated people do several tasks for that product | Do one kind of test for many products | Used by large organizations |

## 06 — MEMORIZE THIS (facts you must know cold)

---

### A. NUMBERED LISTS (with mnemonics)

---

#### 5 CMM levels — mnemonic: "I Really Don't Manage Overtime"

1. **I**nitial — chaotic, individual effort, not repeatable
2. **R**epeatable — basic project management
3. **D**efined — own standard process
4. **M**anaged — data collection & analysis
5. **O**ptimizing — continuous improvement + innovation

#### 5 TMM levels — mnemonic: "I Prefer Increasingly Mature Orgs"

1. **I**nitial — testing after code, ad hoc, no tracking
2. **P**hase Definition — testing/debugging goals, initiate test planning
3. **I**ntegration — establish test group, integrate testing into lifecycle
4. **M**anagement and Measurement — org-wide review, test management, evaluate quality
5. **O**ptimization / Defect Prevention & QC — defect prevention, statistical QC, optimization

#### 7 Principles of Testing — mnemonic: "Please Eat Every Delicious Pizza Carefully, Alright"

1. **P**resence of defects (not absence)
2. **E**xhaustive testing is impossible
3. **E**arly testing
4. **D**efect clustering
5. **P**esticide paradox
6. **C**ontext dependent
7. **A**bsence-of-errors fallacy

#### 6 ISO-9126 characteristics — mnemonic: "Fat Rabbits Usually Eat More Peas"

**F**unctionality · **R**eliability · **U**sability · **E**fficiency · **M**aintainability · **P**ortability (*Security is NOT one of them.*)

#### 5 Views of quality (Kitchenham & Pfleeger) — mnemonic: "My Uncle Makes Pretty Vases"

**M**ystical · **U**ser · **M**anufacturing · **P**roduct · **V**alue-based

#### 3 QA categories — mnemonic: "Prevent, Reduce, Contain" (PRC)

**P**revention (before injection) · **R**eduction (remove faults) · **C**ontainment (limit failures)

#### 4 Testing levels + who — mnemonic: "U-I-S-A: Devs Test Test Users"

**U**nit → Developer · **I**ntegration → Tester · **S**ystem → Tester · **A**cceptance → End-user

#### 3 SqE stages — mnemonic: "PED"

**P**lanning · **E**xecution · **D**ecision (*SQE generic testing process phases: Pre-QA → In-QA → Post-QA.*)

#### Other short lists to keep

- **2 ways of defect prevention:** error source removal · error blocking
- **2 ways of defect containment:** software fault-tolerance (NVP, recovery) · safety assurance
- **3 testing team models:** Vertical · Horizontal · Mixed
- **3 project challenges:** Time · Cost · Scope
- **3 reasons to improve a process:** Quality · Lead Time · Cost

- **4 TPI improvement steps:** determine area → evaluate current → identify next → implement
- **4 process variations:** Waterfall · Iterative · Spiral · Agile (+ Maintenance)

## B. NAMED PEOPLE / MODELS

| Name / Model                           | What it is / who                                                            |
|----------------------------------------|-----------------------------------------------------------------------------|
| <b>McCall</b>                          | Quality <b>Factors and Criteria</b> model                                   |
| <b>Kitchenham &amp; Pfleeger</b>       | The <b>five views</b> of software quality                                   |
| <b>Ilene Burnstein</b>                 | Pioneered the <b>TMM</b> (Testing Maturity Model)                           |
| <b>ISO-9126</b>                        | Most influential quality <b>framework</b> — 6 characteristics               |
| <b>CMM</b>                             | Capability Maturity Model — evaluates <b>development</b> process (5 levels) |
| <b>TMM</b>                             | Testing Maturity Model — evaluates <b>testing</b> process (5 levels)        |
| <b>TPI</b>                             | Test Process Improvement — <b>improves</b> the testing process (4 steps)    |
| <b>ISO 9001</b>                        | Standard based on the <b>manufacturing</b> view (with CMM)                  |
| <b>Naik &amp; Tripathy / Jeff Tian</b> | The two course textbook authors                                             |

## C. FAMOUS INCIDENTS

| Incident         | What happened                                                                                                                                                                          |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Therac-25</b> | Radiation-therapy machine by <b>AECL (1986)</b> ; a <b>software bug</b> caused massive radiation <b>overdoses</b> and deaths.                                                          |
| <b>Ariane 5</b>  | On <b>4 June 1996</b> , the rocket <b>tore itself apart 37 seconds</b> after launch due to a <b>control-software</b> malfunction — the <b>most expensive computer bug in history</b> . |

## D. EVERY "HOW MANY" FACT

| Question                                 | Answer             |
|------------------------------------------|--------------------|
| CMM levels                               | 5                  |
| TMM levels                               | 5                  |
| Principles of testing                    | 7                  |
| ISO-9126 top-level characteristics       | 6                  |
| Views of quality (Kitchenham & Pfleeger) | 5                  |
| QA categories                            | 3                  |
| Testing levels                           | 4                  |
| SqE stages                               | 3                  |
| Generic testing-process phases           | 3 (Pre/In/Post-QA) |
| Ways to do defect prevention             | 2                  |
| Ways to do defect containment            | 2                  |
| Testing team models                      | 3                  |
| TPI improvement steps                    | 4                  |
| Software types (Buy/Build)               | 2                  |
| Project challenges                       | 3                  |
| Ariane 5 seconds to failure              | 37                 |

## E. ONE-LINE DEFINITIONS TO NAIL

- **Error** = human action producing an incorrect result.
- **Fault** = incorrect step/process/data definition in a program.
- **Failure** = inability of a system to perform its required function.
- **Defect** = behavioural deviation from spec (umbrella term = "bug").
- **Reliability** = probability of executing without failure for a specific period of time.
- **Safety** = accident-free. **Accident** = failure with severe consequences. **Hazard** = pre-condition to an accident.
- **Test plan** = high-level; **Test case** = low-level; **Test suite** = collection run until stopping criteria.
- **Regression testing** = reuse existing tests (design none) to check nothing broke.
- **White-box** = structural/implementation; **Black-box** = functional/specification; **Gray-box** = mixed.
- **SQA** = prevent, no execution, verification; **SQC** = detect, executes, validation.